Relatório técnico completo — Front Web QA Challenge

Data do relatório: 3 de novembro de 2026
Projeto: automação E2E da aplicação pública PlayLab
Tecnologia principal: Playwright Test com TypeScript

1. Resumo executivo

O projeto automatiza fluxos selecionados de uma aplicação web pública para demonstrar cobertura funcional, validação de formulário, sincronização com componente assíncrono, acessibilidade básica e execução mobile. A suíte contém **seis testes lógicos independentes**, organizados em cinco arquivos de especificação. Os cinco cenários desktop executam em Chromium e Firefox; o cenário mobile executa em Chromium emulado como Pixel 7.

A configuração centralizada gera relatório HTML e screenshot ao final de cada teste. Os cenários também salvam capturas nos principais passos da jornada para mostrar o estado antes e depois das interações. Em falhas, mantém trace e vídeo. O workflow do GitHub Actions executa a suíte em push, pull request ou solicitação manual e publica os resultados como artefato temporário.

2. Aplicação-alvo e objetivo

A aplicação utilizada é o PlayLab — Playwright Practice Arena, ambiente público de prática com tabelas, formulários, dropdowns e outros componentes.

O objetivo é validar comportamentos observáveis da interface e demonstrar uma estrutura de automação sustentável. Como o ambiente é externo e não controlado pelo projeto, os testes evitam alterar registros da tabela e podem ser afetados por indisponibilidade, rede ou mudanças no site.

3. Escopo implementado

IDs	Grupo	O que o teste verifica
USR-01/02/03/04/07	Jornada da tabela de usuários	Carregamento e cabeçalhos da tabela; busca de usuário existente e inexistente; dados e situação apresentados.
FRM-01	Formulário — cadastro válido	Preenchimento com dados fictícios válidos e confirmação de envio.
FRM-03/06	Formulário — cenário negativo	Rejeição de e-mail inválido, apresentação da mensagem de erro e ausência de confirmação de sucesso.
DYN-01/02	Componente dinâmico — dropdown assíncrono	Estado inicial desabilitado, carregamento das opções e seleção de uma opção disponível.
FRM-07/A11Y-02	Acessibilidade — labels e nomes acessíveis	Associação de labels aos campos obrigatórios e nome acessível do botão de cadastro.
RSP-01	Responsividade — desktop	Execução da cobertura de usuários em viewport desktop de 1280 × 720, em Chromium e Firefox.
RSP-02	Responsividade — mobile	Pesquisa de usuário em viewport Pixel 7 emulado, de 412 × 839, usando Chromium.
Transversal	Relatório e evidências	Geração do relatório HTML e screenshots; retenção de trace e vídeo em falhas; publicação do log e dos artefatos da pipeline por 14 dias.

O total é de seis testes lógicos: cinco testes desktop executados em dois navegadores (dez execuções) e um teste mobile (uma execução), totalizando **11 combinações de teste/projeto**.

Cenários não implementados nesta etapa

Foram documentados como adiados: ordenação, paginação e ação na tabela (USR-05/06/08); submissão vazia, limite de caracteres, reset e teclado no formulário (FRM-02/04/05/08); outros estados dinâmicos (DYN-03 a DYN-08); sobreposição, scroll horizontal e acesso a todas as ações no mobile (RSP-03/04/05); verificações ampliadas de acessibilidade (A11Y-01 e A11Y-03 a A11Y-07); upload, drag-and-drop, iframes, API, carga e stress.

As justificativas e prioridades estão detalhadas na matriz de cenários, na matriz de riscos e na estratégia de testes.

4. Estrutura do repositório

```
.
├── .env.example
├── .github/
│   └── workflows/
│       └── playwright.yml
├── docs/
│   ├── estrategia-de-testes.md
│   ├── matriz-de-cenarios.md
│   ├── matriz-de-riscos.md
│   └── relatorio-completo-do-projeto.md
├── src/
│   ├── data/
│   │   └── registration.ts
│   ├── fixtures/
│   │   ├── capturar-etapa.ts
│   │   └── test.ts
```

```
│   └── pages/
│       ├── delayed-dropdown.page.ts
│       ├── registration.page.ts
│       └── users.page.ts
├── tests/
│   ├── acessibilidade/
│   │   └── cadastro-acessibilidade.spec.ts
│   ├── dinamicos/
│   │   └── menu-suspenso-assincrono.spec.ts
│   ├── formularios/
│   │   └── cadastro.spec.ts
│   ├── responsividade/
│   │   └── usuarios-dispositivo-movel.spec.ts
│   └── usuarios/
│       └── usuarios.spec.ts
├── .gitignore
├── package.json
├── package-lock.json
├── playwright.config.ts
└── tsconfig.json
```

Responsabilidade de cada arquivo

Arquivo	Responsabilidade
`.env.example`	Exemplo de configuração de `BASE_URL`; pode ser copiado para `.env` para apontar a suíte para outro ambiente compatível.
`.gitignore`	Impede versionar dependências, relatórios, resultados de teste, logs locais, autenticação Playwright e arquivos `.env` locais. Mantém `.env.example` permitido.
`.github/workflows/playwright.yml`	Define gatilhos, ambiente Node.js, instalação de dependências e navegadores, execução e upload de relatório/evidências no GitHub Actions.
`docs/estrategia-de-testes.md`	Registra objetivo, escopo, jornadas, riscos, priorização, estratégia de automação, navegadores, pipeline, limitações e próximos passos.
`docs/matriz-de-cenarios.md`	Relaciona IDs, jornada, descrição, tipo, risco protegido, prioridade, decisão de automatizar, justificativa e status.
`docs/matriz-de-riscos.md`	Descreve riscos, impacto, probabilidade, prioridade, cenários relacionados, mitigação e critérios da priorização.
`docs/relatorio-completo-do-projeto.md`	Este relatório explicativo, com a visão consolidada do projeto e de seus arquivos.
`src/data/registration.ts`	Define o tipo `RegistrationData` e os dados fictícios válidos e inválidos usados nos testes de formulário.
`src/fixtures/capturar-etapa.ts`	Cria screenshots numeradas das principais etapas de cada teste e amplia o limite de tempo para acomodar a captura.
`src/fixtures/test.ts`	Estende as fixtures do Playwright com instâncias tipadas dos page objects, fornecidas por teste. Também exporta `expect`.
`src/pages/users.page.ts`	Encapsula acesso à tabela, busca e leitura de célula associada ao cabeçalho. Normaliza indicadores de ordenação no texto dos cabeçalhos.
`src/pages/registration.page.ts`	Encapsula navegação, preenchimento, envio do formulário e acesso às mensagens de erro e sucesso.
`src/pages/delayed-dropdown.page.ts`	Encapsula acesso ao dropdown assíncrono, solicitação de carregamento e seleção de uma opção não vazia. Erra explicitamente se nenhuma opção estiver disponível.
`tests/usuarios/usuarios.spec.ts`	Implementa a jornada selecionada da tabela (USR-01/02/03/04/07).
`tests/formularios/cadastro.spec.ts`	Implementa envio válido e cenário negativo com validação do e-mail (FRM-01 e FRM-03/06). Um `beforeEach` abre a aplicação antes de cada cenário.
`tests/dinamicos/menu-suspenso-assincrono.spec.ts`	Valida o carregamento por estado observável e a seleção do menu suspenso (DYN-01/02).
`tests/acessibilidade/cadastro-acessibilidade.spec.ts`	Valida rótulos e nome acessível do botão (FRM-07/A11Y-02).
`tests/responsividade/usuarios-dispositivo-movel.spec.ts`	Executa a pesquisa na resolução móvel configurada (RSP-02).
`package.json`	Declara scripts, dependências e metadados do projeto.
`package-lock.json`	Fixa a árvore resolvida das dependências para instalação reproduzível com `npm ci`.
`playwright.config.ts`	Centraliza URL-base, projetos de navegador, resolução, workers, retries, reporter e coleta de screenshots, traces e vídeos.

Arquivo	Responsabilidade
`tsconfig.json`	Configura compilação/verificação TypeScript estrita sem emitir arquivos e inclui configuração, código de suporte e testes.

5. Como o código se organiza

O fluxo geral de execução é:

1. O Playwright lê `playwright.config.ts` e descobre os projetos e testes.
2. Cada teste recebe uma `page` isolada e as fixtures tipadas definidas em `src/fixtures/test.ts`.
3. O teste chama o page object correspondente em `src/pages/`, que concentra seletores e interações da página.
4. As asserções Playwright verificam estados visíveis, valores e mensagens observáveis.
5. O reporter HTML e as opções de evidência guardam os resultados em `playwright-report/` e `test-results/`.

Essa divisão deixa os arquivos de teste focados na intenção do cenário, e os page objects focados em interações reutilizáveis. Os dados fictícios de cadastro ficam fora da lógica de interação.

6. Configuração e execução local

Pré-requisitos

- Node.js 20 ou superior; a pipeline usa Node.js 22.
- npm e acesso à internet.
- Navegadores Playwright correspondentes aos projetos executados.

Instalação

Na raiz do repositório:

```
npm.cmd ci
npx.cmd playwright install chromium firefox
```

Comandos

```
npm.cmd test                # suíte completa em todos os projetos
npm.cmd run test:chromium   # cenários desktop em Chromium
npm.cmd run test:firefox    # cenários desktop em Firefox
npm.cmd run test:mobile     # cenário mobile em Chromium emulado
npm.cmd run test:ui         # interface interativa do Playwright
npm.cmd run typecheck       # verificação TypeScript sem emissão de arquivos
npm.cmd run report          # abre o relatório HTML existente
```

No PowerShell do Windows, use os sufixos `.cmd` para evitar o bloqueio de execução do `npm.ps1` ou `npx.ps1`. Em outros terminais, os comandos podem ser escritos como `npm` e `npx`.

Para configurar outro ambiente compatível, ajuste `BASE_URL` no arquivo `.env` (criado localmente a partir de `.env.example`) ou defina a variável antes da execução. O arquivo `.env` não deve ser versionado.

7. Navegadores, resoluções e compatibilidade

Projeto Playwright	Navegador	Resolução/perfil	Seleção de testes
`chromium`	Chromium desktop	1280 × 720	Todos os cenários desktop; exclui a especificação mobile.
`firefox`	Firefox desktop	1280 × 720	Todos os cenários desktop; exclui a especificação mobile.
`mobile-chromium`	Chromium com emulação Pixel 7	412 × 839	Especificação `tests/responsividade/`.

Chromium cobre o navegador principal; Firefox fornece um segundo mecanismo de navegador. A cobertura mobile fica separada em Chromium emulado para limitar o número de combinações. A emulação não substitui aparelho físico.

8. Qualidade e decisões de automação

- Testes independentes e sem compartilhamento intencional de estado entre cenários.
- Títulos em português, com IDs do case e comportamento esperado.
- Asserções ligadas ao resultado funcional, não apenas à execução de cliques.
- Seletores prioritariamente por `data-testid`; seletores semânticos (`getByLabel`, `getByRole`) na validação de acessibilidade.
- Leitura de células por cabeçalho, ignorando os símbolos de ordenação (↕, ↑, ↓) no nome acessível do cabeçalho.
- Sincronização por expectativas auto-retry, como visibilidade, habilitação e quantidade de linhas; sem `waitForTimeout`.
- Dados de cadastro fictícios e determinísticos; a tabela pertence ao ambiente público e não é alterada pelos testes implementados.
- Erros explícitos quando cabeçalho ou opção de dropdown necessária não é encontrada; sem captura genérica ou fallback silencioso.
- Dois workers para equilibrar duração e carga no site externo.

9. Relatórios, screenshots, traces e logs

- Reporter HTML: `playwright-report/`.
- Screenshots: `screenshot: 'on'` guarda uma captura ao fim de cada teste; `src/fixtures/capturar-etapa.ts` também grava imagens numeradas das principais etapas em `test-results/`.
- Trace e vídeo: `retain-on-failure`, preservados em falhas para diagnóstico.
- Log da pipeline: `playwright-test.log`, mostrado na saída do workflow e incluído no artefato.
- Retenção de artefatos GitHub Actions: 14 dias.
- `test-results/`, `playwright-report/` e o log são ignorados pelo Git para evitar versionar dados temporários ou pesados.

Na interface do GitHub, os artefatos podem ser baixados em **Actions > execução do workflow > Artifacts**.

10. Pipeline GitHub Actions

O workflow `.github/workflows/playwright.yml` roda em `push`, `pull_request` e `workflow_dispatch`.

Passos automatizados

1. Checkout do repositório.
2. Configuração do Node.js 22 com cache npm baseado no lockfile.
3. Instalação reproduzível com `npm ci`.
4. Instalação de Chromium e Firefox, incluindo dependências do sistema operacional.
5. Execução da suíte, mantendo o código de falha do Playwright com `set -o pipefail`, enquanto `tee` salva e exibe o log.
6. Upload do relatório, screenshots, traces, vídeos disponíveis e log. O upload usa `if: always()` para ocorrer também quando uma etapa anterior falha.

Em execução manual, os inputs permitem selecionar `all`, `chromium`, `firefox` ou `mobile-chromium` e filtrar os testes por ID ou texto do título. Em push e pull request, todos os projetos configurados são executados.

11. Histórico de alterações e validação

O projeto evoluiu de uma suíte inicial para uma estrutura com page objects, fixtures, dados centralizados, documentação de estratégia e riscos, matriz de cenários, compatibilidade desktop/mobile e workflow GitHub Actions. Também foram alinhados os títulos e comentários dos testes em português aos IDs do case.

Na execução mais recente registrada antes da correção do page object, a jornada da tabela falhou em Chromium e Firefox ao procurar a coluna cujo cabeçalho acessível aparecia como `Status ↕`, embora o código esperasse exatamente `Status`. O page object foi ajustado para remover os indicadores de ordenação antes da comparação. **A alteração foi feita após aquela execução e ainda não foi validada por uma nova execução dos testes.** Portanto, este relatório não declara a suíte atualmente aprovada.

Verificações de documentação e alterações foram acompanhadas por `git diff --check`. Por orientação da usuária, os testes não são executados automaticamente a cada alteração; execute-os quando desejar validar a correção.

12. Limitações conhecidas

- A aplicação é pública e pode ficar indisponível ou mudar sem aviso.
- Dados da tabela não são controlados localmente pelo projeto.
- O teste mobile é uma emulação e cobre a busca, não todos os aspectos visuais e ações.
- A verificação de acessibilidade cobre labels e nome acessível básico; não é uma auditoria WCAG completa.
- A validação não substitui testes manuais de teclado, leitor de tela, contraste ou dispositivos reais.
- Cenários adiados podem ser adicionados conforme risco, estabilidade e prioridade.

13. Próximos passos recomendados

1. Executar novamente os testes desktop em Chromium e Firefox para confirmar a normalização do cabeçalho `Status`.

2. Revisar relatório HTML e screenshots; investigar qualquer falha externa separadamente de regressão.
3. Usar `npm run typecheck` para verificar tipos se houver mudanças no TypeScript.
4. Considerar ampliar cobertura de ordenação, paginação, ação na tabela, navegação por teclado e layout mobile.
5. Avaliar axe-core com triagem dos resultados e complementar a acessibilidade com revisão manual.

14. Links internos

- README do projeto
- Estratégia de testes
- Matriz de cenários
- Matriz de riscos
- Workflow do GitHub Actions